Distributing standards — an artifact your team defines

Your team agreed on some rules — how errors are handled, what "done" means, which things an AI agent must never touch. You wrote them down in one repository. **You have more than one repository.** A standard that lives in one repo isn't a standard yet; it's a local custom.

This document is about the gap between those two states: **how a rule written once reaches every repo, and how anyone can tell whether the copy in front of them is current.** That question has an answer your team chooses — and the choice itself should be written down, owned by a named person, and revisited when it stops fitting. In other words, it is one more of the governance documents Lab 2 taught, sitting one level up: it applies to *all* your repos, not any single one.

This document is not the answer. It's the options, honestly traded off, and a template at the end for recording what you chose. If you do only two things: do the **Crawl** step below (about thirty minutes), and fill in the template.

> **Why you and not your instructor.** Choosing the mechanism, the starting point, and the owner *is* the act of turning team habits into organisation-wide standards. A team that receives its distribution model has been handed a standard rather than adopting one — and the whole point of the governance work is that you own it.

Four words this document leans on

Term	What it means here
Standard	Any rule your team wrote down and expects to hold: an ADR, an NFR, a recipe, a guardrail. (Lab 2.a defines each.)
Drift	The copies no longer match — repo A has the new rule, repo B has last month's, and nothing tells you
Pin	A repo records *which version* of the standards it follows (a tag like `v1.4.0`), so staleness is checkable
Enforce	Something mechanical — a hook, a CI check — refuses violations. A document informs; a gate enforces

---

Start where it costs nothing

The trap is designing the mature version first. Every rung below works, and each is useful on its own; you can stop at any of them and still be better off than today.

Crawl — one repo, two lines (~30 minutes)

One person's afternoon, no tooling, and you are better off than today:

1. Create a repo called `sdlc-governance` with four folders: `adr/`, `nfr/`, `best-practices/`, `templates/`.
2. Seed it by **promoting** your best existing standards — the Lab 2 NFR is exactly right. The governance repo becomes the authoritative home; the project repo keeps a **pointer** (its CLAUDE.md trigger row now aims at the governance copy). One home per fact, and nothing the labs wired up breaks.
3. In every project's `CLAUDE.md`, add two lines: where the governance repo is, and "before doing X, read Y" for the rules that matter most.
4. Name the owner and fill in the template at the end of this document.

People read the standards on GitHub. That's it — that's the whole mechanism at this rung.

Good at: it exists today; one source of truth; zero adoption cost. **Can't:** nothing is local, nothing is enforced, and nobody knows whether they've read the current version.

Walk — make it queryable, and make staleness visible

Crawl's two weaknesses are that finding a rule means remembering which document it lives in, and that a copied rule can silently go stale. Walk fixes both, with two independent steps you can do in either order.

Step 1 — make it queryable

The goal: from inside any project, ask the governance repo a question and get the right document — without having to remember where anything lives. Three ways to get there; pick whichever your team can actually get installed. The step is about queryability, not about a specific tool.

Option A — semble *(semantic: finds rules by meaning)*

semble (PyPI, MIT) describes itself as *"fast and accurate code search for agents"* — so it is worth being clear about what it actually is, because searching a governance repo is a side application rather than its purpose.

It is a **retrieval layer for coding agents**, not a developer-facing grep replacement:

- **Search** over a repo — local path or a remote `https://` URL — in hybrid, semantic, or BM25 modes, so "the rule about logging" finds the right document even when that document never uses the word "rule". Index builds on first run, caches, and invalidates itself when files change.
- `find-related` — similarity search from a known `file:line`.
- `semble install` — wires itself into your coding agents as an MCP server (MCP is the protocol agents use to call external tools), as instructions in the agent's context file, and as a subagent.
- `semble savings` — reports tokens saved versus having the agent read files directly. That is the actual pitch: an agent that searches instead of reading burns far less context.

Which is useful to know for two reasons. If your team already runs it for code, pointing it at a governance repo costs nothing extra. And if you don't, adopting a whole agent-retrieval tool purely to search a dozen markdown files is disproportionate — Option B below is the right size for that.

```
# install once (needs uv — https://docs.astral.sh/uv/)
uv tool install semble
semble --help                        # confirm it is on PATH

# then, from any project, with nothing cloned:
semble search "logging convention" https://github.com/<org>/sdlc-governance --content
        docs

# or run it without installing anything:
uvx --from semble semble search "logging convention" <repo-url> --content docs
```

Two things that will otherwise waste your afternoon:

- **`--content docs` is required.** The default indexes code only, so without it a markdown-only governance repo returns nothing and looks broken. `--content` needs **semble ≥ 0.5.0** (`uv tool list` to check, `uv tool upgrade semble` to fix).
- **Run it from the shell, not as an MCP tool.** The MCP interface takes no content parameter, so it silently indexes code only. Same trap, different route.

Option B — a sparse clone plus grep *(literal matching; always works)*

The one that always works — offline, on private hosts, on any repo, with tools you already have:

```
# once: a docs-only checkout, no blobs for anything else
git clone --depth 1 --filter=blob:none --sparse \
  https://github.com/<org>/sdlc-governance ~/gov
git -C ~/gov sparse-checkout set docs

# then, whenever:
rg -i "logging" ~/gov/docs          # or grep -ri "logging" ~/gov/docs
git -C ~/gov pull                   # refresh
```

Trade-off: you keep a local copy (so it can go stale — `pull` before you trust it), and matching is **literal, not semantic**, so you need to guess the word the document uses. Nothing to install and nothing to go wrong. **If a team adopts only one thing from this rung, adopt this.**

Option C — GitHub's own search *(for one-off lookups)*

For a one-off "where is that rule", the repo's search box on github.com works with no tooling at all and uses GitHub's current search engine.

`gh search code "<term>" --repo <org>/sdlc-governance` looks like the scriptable version of the same thing, **but verify it before relying on it.** It runs on GitHub's legacy code-search API, whose index has coverage gaps — on our own training repo it returns **zero results with exit code 0** for terms that demonstrably exist in the files. Silent empty results are worse than an error, so test it with a string you know is there before you put it in a runbook.

> **Whichever you choose, write it down in the artifact at the end of this document.** "How a developer finds the rule" is exactly the kind of thing that lives in one person's shell history instead of the standard.

Step 2 — make staleness visible (pinning)

Sometimes the docs genuinely need to live inside each project — agents read them without extra tooling, and they work offline. The moment you copy them, drift becomes possible, so the copy must carry a **pin**: a record of which version it is, checkable by a script.

```
// package.json in YOUR project repo — the tools/ script is yours to write (~20
        lines)
"scripts": {
  "gov:sync": "giget gh:<org>/sdlc-governance/docs#v1.4.0 docs/governance --force",
  "gov:check": "node tools/check-governance-version.mjs"
}
```

(`giget` is a small npm tool that downloads one folder from a git repo, no history, no install — `npx giget` just works. The `#v1.4.0` is the pin: you are asking for a *named version*, not "whatever is newest".)

How it stays honest: `gov:sync` writes the tag into a `docs/governance/VERSION` file, and a pre-commit hook **verifies** the pin matches and warns when it doesn't. Publishing the docs as an npm package instead gets you the same result with `npm outdated` reporting staleness for free.

Good at: cheap; turns silent drift into visible drift. **Can't:** still no enforcement; N copies to keep in step.

Run — version the enforcement, serve the docs, template new repos

Everything so far distributes documents, and documents only inform. The Run rung is where rules start being **enforced** — and where the tooling starts needing an owner. Four independent moves; adopt them one at a time, in any order, as the need appears:

Move	Mechanism
Guardrails become versioned	A **Claude Code plugin from a private marketplace** — the only mechanism that versions the *enforcement* layer rather than the documents. See the advanced subsection below
Docs get identity and enumeration	A small **docs MCP server** — a local service your coding agent queries for search / get-by-id / list. Committed in `.mcp.json`, so every teammate gets it automatically. Graduate here when you need "give me ADR-0042" deterministically
Rules become gates	CI checks for the mechanically checkable clauses; a documented bypass for the rest
New repos start compliant	A **template repo** with the hooks, CI gates, and docs already wired in — so a new project starts compliant instead of retrofitting. (This training repo is itself one)

Advanced — ship the enforcement layer as a Claude Code plugin

Advanced. Do not start here. This is the right answer once you have guardrails worth versioning, and premature if you don't.

Everything above distributes *documents*. Documents inform; they don't enforce. A **plugin** is the only mechanism that versions the enforcement layer itself — a self-contained, versioned unit bundling `hooks/`, `agents/`, `skills/`, slash commands, MCP config, and default settings, installed and updated like a dependency.

The mechanics, in three files. A marketplace repo lists your plugins:

```
// .claude-plugin/marketplace.json  in <org>/claude-plugins
{
  "name": "acme-standards",
  "owner": { "name": "Acme Platform" },
  "plugins": [{ "name": "sdlc-guardrails", "source": "./plugins/sdlc-guardrails" }]
}
```

The plugin declares its own version — bump it to release, or omit it and every commit counts as a new version:

```
// plugins/sdlc-guardrails/.claude-plugin/plugin.json
{
  "name": "sdlc-guardrails",
  "description": "Org guardrails: hooks + review agent",
  "version": "1.2.0"
}
```

Each consumer repo asks for it once, in committed project settings:

```
// .claude/settings.json  in every project repo
{
  "extraKnownMarketplaces": {
    "acme-standards": { "source": { "source": "github", "repo": "<org>/claude-
        plugins" } }
  },
  "enabledPlugins": { "sdlc-guardrails@acme-standards": true }
}
```

Anyone who trusts the folder is prompted to install it. Publish a new deny rule, bump the version, and it reaches everyone on their next session — no PR in twelve repos, no copy-paste, no drift.

Why it's advanced, honestly. Four real costs:

- **It needs an owner and a release process.** A plugin that silently changes what the team's agents refuse to do is a change-management problem, not a file copy. Version bumps are announcements.
- **It's Claude Code-specific.** Humans reading docs, and any other tooling, still need one of the channels above. Plugins are for the enforcement layer, not the whole corpus.
- **Every consumer needs access** to the marketplace repo — trivial internally, a real question with contractors.
- **You need something worth shipping.** One hook is not a plugin; it's a file.

The governance upside worth knowing: managed settings can restrict which marketplaces are permitted at all (`strictKnownMarketplaces`), so an org can require that plugins come only from its own. That is the strongest available answer to "how do we know what our agents are running?"

Before this, do the crawl rung. A governance repo plus two lines in a context file, this afternoon. Get to a plugin when you have three or more guardrails that more than one repo needs.

The trade-off table

How to read it: **Drift** = can the copies silently stop matching? **Enforces?** = does anything mechanical refuse a violation, or does this only inform? **Versioned?** = can a repo say *which* version of the standards it follows?

Mechanism	Effort	Drift	Enforces?	Versioned?	Best when
Governance repo + a CLAUDE.md pointer	~30 min	Invisible	No	No	Day one, always
Walk step 1 — queryable	0–5 min	None — always current	No	n/a	Any corpus size (options A/B/C above)
Walk step 2 — pinned local copies	1–2 hrs	**Visible**	No	Yes	Docs must live inside each project
Push-based sync PRs (see below)	~half day	None	No	Yes	Many repos; audit trail matters
Plugin + private marketplace	~half day + **an owner**	None	**Yes**	**Yes**	Guardrails, once 3+ exist that several repos need
Docs MCP server	1–2 days	None	No	Yes	Corpus past ~20 docs

One row above hasn't been described yet: **push-based sync PRs** inverts Walk step 2 — instead of each repo pulling, a GitHub Action in the governance repo opens a pull request in every consumer repo whenever a standard changes (e.g. `repo-file-sync-action`). Nothing drifts silently and every change arrives reviewable, at the cost of a PR to review in each repo, each time.

Two rules that hold at every rung

1. **A pre-commit hook verifies the pin; it never fetches.** Network access on every commit breaks offline work and mutates the tree mid-commit. Pull explicitly, or on install.
2. **Whatever you can't enforce, you write down — including who accepted the risk.** Same discipline as the marked unenforceable clauses in an NFR (Lab 2.a's marking convention).

Record the decision

Copy this into your governance repo as `docs/process/standards-distribution.md`. Two paragraphs and a table beats a diagram nobody maintains.

```
# How our standards are distributed

**Owner:** <a person, not a team>
**Rung:** crawl | walk | run
**Decided:** YYYY-MM-DD · **Next review:** YYYY-MM-DD

## Where standards live

<repo URL, and what belongs in it vs what stays project-local>

## How a project gets them

<the mechanism, with the exact command or config a developer runs>

## How you know you're current

<the pin, the marker file, the check — or "you don't, yet", honestly>

## What is enforced vs advisory

| Standard | Enforced by | Advisory only |
| -------- | ----------- | ------------- |
|          |             |               |

## Bypass

<how to deviate, who approves, where it gets recorded>

## Not doing yet

<the rungs you've deliberately skipped, and what would trigger revisiting>
```

The two fields people skip are the two that matter. *Owner* — a named person, because an unowned distribution mechanism decays into copies that no longer match. And *Not doing yet* — because a deliberate omission is a decision, while an undocumented one just looks like neglect.

Related

- `enforcing-a-convention.md` — turning one rule into something that refuses to be violated
- `../labs/lab-2a-handout.md` — what an ADR / NFR / recipe / rules file each *is*, and how to tell them apart. Read this first if those names are new to you

- `../labs/lab-2b-handout.md` — the enforcement layers (hooks, gates, bypasses); ends by handing this artifact to your team